

CSC3003S Capstone Project — Stage One

Goals (Scope) [21 Marks]

Project Abbreviation and Name	UAV-Path – Visualizing and measuring optimal UAV flight paths
Client/Supervisor + email	Patrick Marais <patrick@cs.uct.ac.za>
Tutor + email	Raaziq Parkar <prkraa002@myuct.ac.za>
Date	08 August 2025
Team Nickname	UAV Team
Team Members	Student Number, First and Last Name, email
	BLYKEN007, Kenneth Baloyi, blyken007@myuct.ac.za
	MKHXMAX003, Max Mkhabela, mkhmax003@myuct.ac.za
	NXXLET001, Lethabo Neo, nxxlet001@myuct.ac.za
Overall purpose and stakeholders [5]	The purpose of this project is to develop a system for visualizing and measuring optimal flight paths for Unmanned Aerial Vehicles (UAVs) using interactive 3D rendering and simulation techniques. The project aims to bridge the gap between manual flight path planning and practical, real-world application by providing a visual and analytical system that can simulate and evaluate UAV navigation strategies in various environments. This system will assist in identifying the most efficient, safe, and cost-effective flight paths by integrating basic optimization algorithms with 3D visualization libraries such as PyRender and Plotly. Depending on the complexity and application needs, additional rendering and simulation libraries such as Open3D or VTK may be explored to enhance realism and scalability. By visually modelling flight paths in a 3D space, this system can assist in decision-making for path planning, risk avoidance including collision or restricted zones, and energy conservation, particularly in environments where manual path assessment is infeasible or error prone. Beyond its educational and research value, this project has potential applications in fields like Logistics, Disaster response, Agriculture, and Surveillance, where UAVs are increasingly deployed for data collection, delivery, and monitoring tasks. By focusing on visual measurability and interactivity, the system is not just a back-end optimizer but also a tool for communication, demonstration, and operational planning. Stakeholders for this system excluding us the development team include: • Our supervisors and tutors because they are responsible for the overall evaluation of the system ensuring it aligns with the assessment criteria and the requirements. • Users: Including UAV operators in general and airline pilots possibly because they are target users for this system that will assist them with flight path planning, flight safety and flight path optimization. • Researchers and engineers doing professional work such as geospatial analysis. They may use or adapt the system for experiments or integrate

	it into broader research models. Public and private sector organizations may find use in real-world deployment situations like infrastructure planning and land surveying.
SMART Goals [5]	Start by listing the functions and features that will be available to users. That is, what are all the things a user can do? Then list all goals: Specific, Measurable, Agreed Upon, Realistic, Trackable. 1.Goal: Load 3D Scene and Model: Functions and features 3D scene visualisation Flight trajectory display User interface First-Person View simulation Camera Controls Export Video recordings Smart Goals: 1.Goal: Load 3D Scene and Model: Specific: The system will load a 3D model in a common format (JSON, OBJ or PLY) and initialize a 3D scene. The scene will contain the loaded structure, ground reference plane, and co-ordinate axes for orientation. Basic camera controls for navigating the scene (X, Y, Z translation and yaw/pitch rotation) will be implemented. Measurable: This goal is achieved when the provided 3D models can be successfully loaded and rendered without errors, and a user can navigate the scene using the implemented camera controls. Agreed Upon: This goal directly implements Core Requirements 2 and 4 from the project specification. Realistic: This is achievable using standard libraries like Open3D or Trimesh, which are suggested in the project are designed for most of the heavy lifting. Timely: This will be completed and demonstrated by the end of the project's second sprint. 2. Goal: Implement trajectory parsing and path visualization: Specific: We will create a module to parse flight trajectory data from the specified JSON file structure, including the cameras array and waypoint. The system will then visualize the flight path in the 3D scene as a smooth curve using spline interpolation, with directional indicators showing the flight heading. Measurable: Success of the feature is measured by the system's ability to correctly parse all provided sample trajectory files and render a path that connects the waypoints in the exact sequence defined by the waypoint array. Agreed Upon: This goal meets Core Requirements 3 and 5 from the project specification. Realistic: Python's built-in JSON library and numerical libraries like

NumPy/SciPy make the parsing and mathematical interpolation operations easy.

- **Timely:** This feature will be completed by the end of the third sprint.

3. Goal: Create an interactive waypoint selection and First-Person View (FPV):

- **Specific:** The system will display markers at each waypoint that show the camera's orientation (like yaw and pitch). A user must be able to select a waypoint, which will be visually highlighted. When selected, a separate FPV preview window will display the rendered view from that waypoint's exact position and orientation.
- **Measurable:** This will be achieved when clicking a waypoint marker successfully highlights it and the FPV window immediately updates. The orientation shown in the FPV preview must match the yaw and pitch data for the selected waypoint in the JSON file.
Agreed Upon: This goal directly corresponds to Core Requirements 6 and 7 in the project brief.
- **Realistic:** The READ.me file notes that PyRender is a suitable library for generating the FPV preview images, thus making this goal very achievable.
- **Timely:** This interactive functionality will be completed and demonstrated by the end of the fourth sprint.

4. Goal: Create a multi-trajectory (flight path) comparison and metrics display

- **Specific:** We will implement functionality to load, display, and analyze multiple flight paths simultaneously. The system will calculate and display key comparative metrics for each path, including total path length, total vertical distance, and estimated flight duration, in a side-by-side comparison interface for user analysis.
- **Measurable:** This goal will be met when the system can display at least two paths concurrently and the calculated metrics are displayed in the UI. The flight duration metric must correctly use the specified constant cruising speed and hover time.
- **Agreed Upon:** This directly implements the specifications of Core Requirement 9.
- **Realistic:** The required metric calculations are straightforward algebra, these be compatible with a library like NumPy.
- **Timely:** The comparison features will be completed by the end of the fifth sprint.

5. Goal: Create a simulation video recording and export

- **Specific:** The system will include a feature to record flight simulations as video files or image files. The implementation will support customizable resolution and framerate settings. The exported video will overlay trajectory information and position indicators for better context. The user will be able to snap shots of different angles from a given waypoint too
- **Measurable:** Success is measured by the system's ability to export a playable video file or an image like MP4 or png (where the resolution and framerate match the user's selections).
- **Agreed Upon:** This goal corresponds to Core Requirement 12.

	• Realistic: This is a common feature in simulation software and is achievable using widely available Python libraries such as OpenCV or imageio to capture rendered frames and compile them into a video. • Timely: This feature will be completed ahead of final project submission.
Inputs, outputs, and performance [5]	What will the inputs from the user be? • Smooth path representation in the 3D scene displayed on screen • Visual waypoint markers at each trajectory point • Display the rendered view to simulate the UAV's perspective • display comparative metrics including total path length, total vertical distance, and estimated flight duration • Flight simulation recordings as video files What will the outputs to the user be? • Smooth path representation in the 3D scene displayed on screen • Visual waypoint markers at each trajectory point • Display the rendered view to simulate the UAV's perspective • display comparative metrics including total path length, total vertical distance, and estimated flight duration • Flight simulation recordings as video files Are there response time requirements, etc.? • UAV hover time 3 seconds • UAV cruising speed 60 frames per second • Mouse/keyboard event (10 ms)
Resources and Constraints [3]	What resources apart from the project team are needed? • Good performance computer that has a GPUs to support 3D rendering • Multicore CPU to allow for multi-threading • 3D rendering libraries such as PyRender, Open3D, or Trimesh • Gitlab for version control • VS code as IDE In what ways will the software be limited? • Lack of GPU will significantly slow down 3d rendering leading to the overall software being very slow. • Fewer CPU cores will limit the speed of the leading to some parts of the software to be irresponsive when CPU heavy processes are running. What factors will present a challenge? • Implementing a multi-threaded architecture, which may be very hard to implement and may come with risks such as race conditions and deadlock making the software unreliable or unusable. Memory? • Since the will be 3D rendering the system will require more memory, at least 8GB of memory, but most preferably 16GB Hardware? • Since this will be a simulation a computer with a GPU and multi-

	core CPU will be sufficient Other software? Environmental conditions (e.g. uses a camera, cannot work in the dark)? • This is a simulation hence there is a very limited effect when it comes to environmental conditions
Feasibility [3]	Performance and UI Responsiveness: 3D rendering is computationally expensive. A challenge will be implementing multi-threaded architecture needed to not block the user interface, especially in a high-level language like Python unlike Java and C++, a. Failing to manage this could make the app feel slow. Technical Complexity of 3D graphics: As the libraries will handle and simplify 3D rendering, tasks like correctly representing camera positions for waypoint markers, generating smooth spline interpolations and managing an interactive 3D scene still need some 3D mathematics and graphics programming concepts. Worst cases might affect our machines GPUs. Integration of Components: Since systems needs integrating many different components: a 3D scene viewer, a separate FPV preview window, a waypoint data list, and a control panel. These components need to communicate effectively and update in real-time is an integration challenge. Scope Management: a feasibility concern is managing time effectively for Core requirements and Mastery Work options. There is a risk of getting stuck on advanced feature at the expense of core system.

Instructions:

1. Replace any text on the right-hand-side column with information about your project.
2. Notice that there is no indication of how the inputs are transformed into outputs. There is no schedule either. That would be too much detail at this stage. We want only the “what”, not the “how” or the “when” just yet.
3. If something is not applicable to your project, state this and explain why and provide some other relevant and appropriate information. Don’t leave any blanks.
4. Submission is on Amathuba. Remove these instructions from your submission.

Follow-up

1. Bring this document to your next meeting with your client (and email it to them beforehand) and ask if it adequately represents the project. The “overall purpose” and “goals” sections are most important in this regard.
2. Resolve and note the solutions to any issues you might have about feasibility.
3. If the client is satisfied then please **ask them to sign off on this**. Otherwise arrange a revision and time to meet to sort out the issues.
4. *Note that your mark will be determined by your first submission.*